

* Unit IV: CPU scheduling & Algorithms [4M]

- What is Scheduling?
- It is an important function of an operating system. The process scheduling is activity of process manager that handles removal of running process from CPU & selection of another process on basis of particular strategy.

* Difference: FCFS & SJF

FCFS

1) FCFS executes the processes in order in which they arrive i.e. process that arrives first is executed first.

2) It is non-preemptive in nature

Does not suffer from starvation

Waiting time is large

SJF

SJF executes processes based upon their burst time i.e. in ascending order of their burst times.

It is non-preemptive but its preemptive version is also called Shortest Remaining Time First (SRTF)

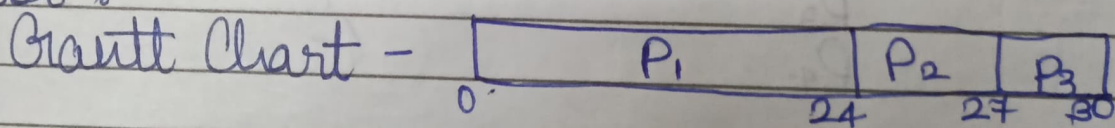
Suffers from starvation

Waiting time is less as compared to FCFS

- FCFS Algorithms
 - First Come first serve CPU scheduling algorithm shortly known as FCFS is first algorithm of CPU process scheduling.
 - Supports non-preemptive & preemptive CPU scheduling algorithms.
 - Tasks are easy to implement & user
 - General wait time is quite high
 - eg: Buying tickets from counter.
 - Easy implemented with FIFO queue but ^{poor in} performance
- ★ Examples

①	Process	Burst Time ^(Time req. to execute a process)
	P ₁	24
	P ₂	3
	P ₃	3

Solⁿ:

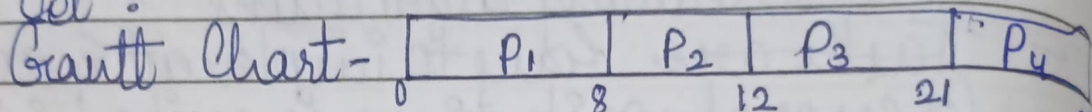


Waiting time for P₁ = 0, P₂ = 24, P₃ = 27

$$\begin{aligned}
 \text{Avg waiting time} &= \frac{(\text{No. of waiting time for processes})}{\text{Total no. of process}} \\
 &= \frac{(0 + 24 + 27)}{3} \\
 &= 17 \text{ ms}
 \end{aligned}$$

$$\begin{aligned}
 \text{Avg turnaround time} &= \frac{(\text{Burst time} + \text{waiting time of all processes})}{\text{No. of process}} \\
 &= \frac{(24 + 27 + 30)}{3} \\
 &= 27 \text{ ms}
 \end{aligned}$$

② Process	Burst Time
P ₁	8
P ₂	4
P ₃	9
P ₄	5

Solⁿ:

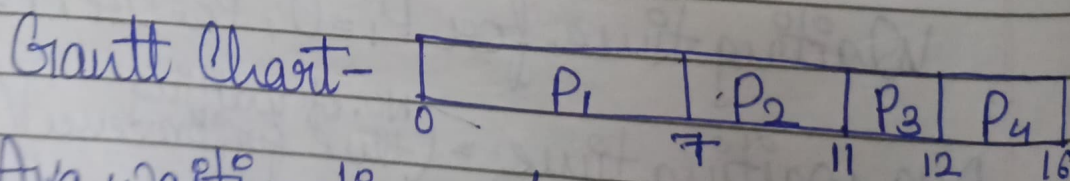
$$\text{Avg waiting time} = (0 + 8 + 12 + 21) / 4$$

$$= 10.25$$

$$\text{Avg turnaround time} = (8 + 12 + 21 + 26) / 4$$

$$= 16.75$$

③ Process	Burst Time
P ₁	7
P ₂	4
P ₃	1
P ₄	4

Solⁿ:

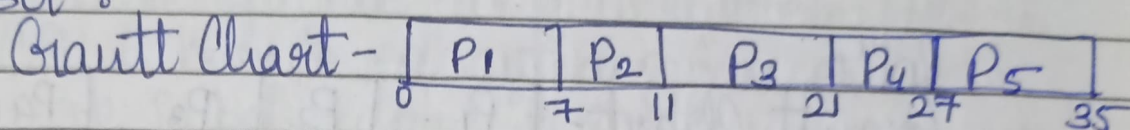
$$\text{Avg waiting time} = (0 + 7 + 11 + 12) / 4$$

$$= 7.5$$

$$\text{Avg turnaround time} = (7 + 11 + 12 + 16) / 4$$

$$= 11.5$$

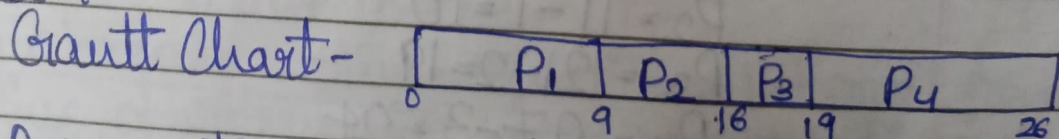
④ Process	Burst Time
P ₁	7
P ₂	4
P ₃	10
P ₄	6
P ₅	8

Solⁿ:

$$\text{Avg waiting time} = (0 + 7 + 11 + 21 + 27) / 5 = 13.2$$

$$\text{Avg turnaround time} = (7 + 11 + 21 + 27 + 35) / 5 = 20.2$$

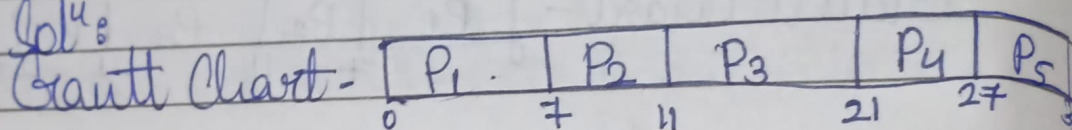
⑤ Process	Burst Time
P ₁	9
P ₂	7
P ₃	3
P ₄	7

Solⁿ:

$$\text{Avg waiting time} = (0 + 9 + 16 + 19) / 4 = 44 / 4 = 11$$

$$\text{Avg turnaround time} = (9 + 16 + 19 + 26) / 4 = 70 / 4 = 17.5$$

⑥ Process	Arrival time	Burst time
P ₁	0	7
P ₂	1	4
P ₃	2	10
P ₄	3	6
P ₅	4	8

Solⁿ:

Waiting time = Start time - Arrival time

$$P_1 = 0 - 0 = 0$$

$$P_2 = 7 - 1 = 6$$

$$P_3 = 11 - 2 = 9$$

$$P_4 = 21 - 3 = 18$$

$$P_5 = 27 - 4 = 23$$

$$\text{Avg waiting time} = (0 + 6 + 9 + 18 + 23) / 5$$

$$= 56 / 5 = 11.2$$

Turnaround time = End time - Arrival time

$$P_1 = 7 - 0 = 7$$

$$P_2 = 11 - 1 = 10$$

$$P_3 = 21 - 2 = 19$$

$$P_4 = 27 - 3 = 24$$

$$P_5 = 35 - 4 = 31$$

$$\text{Avg turnaround time} = (7 + 10 + 19 + 24 + 31) / 5$$

$$= 91 / 5 = 18.2$$

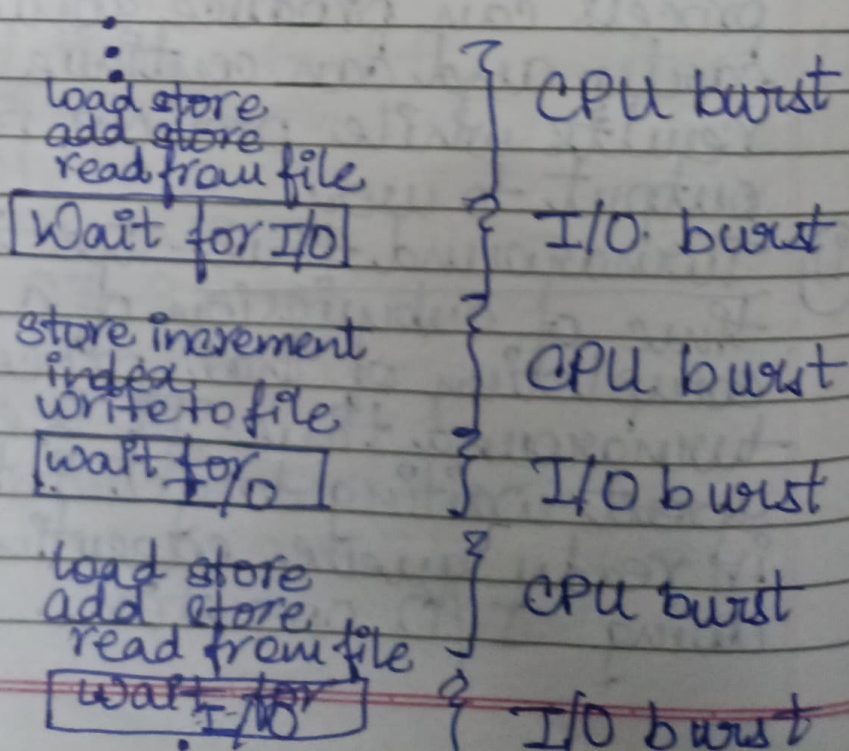
▪ Scheduling criteria

- (i) CPU utilization - In multiprogramming the main objective is to keep CPU as busy as possible. CPU utilization can range from 0-100 percent.
- (ii) Throughput - It is the number of processes that are completed per unit time. It is measure of work done in system. When CPU is busy in executing processes, then work is being done in system. Throughput depends on execution time required for any process. For long processes, throughput can be one process per unit time whereas for short processes it may be 10 processes per unit time.
- (iii) Response time - Time period from submission of request until first response is produced is called as response time. It is time when system responds to process request not completion of process. In the system, a process can produce some output fairly early and can continue computing new results while previous results are being output to user.
- (iv) Turnaround time - The time interval from time of submission of a process to time of completion of that process is called as turnaround time. It is sum of time period spent waiting to get into memory, waiting in ready queue, executing with CPU & doing I/O operations.

- STFS algorithm
- Process with shortest expected processing time is assigned to CPU first.
- It is ~~non-preemptive~~ & preemptive algorithm
- CPU and I/O burst cycles

- CPU burst cycle - It is time period when process is busy with CPU.
- I/O burst cycle - It is time period when process is busy in working with I/O resources.
- CPU bound processes - The process which spends more time in computations or with CPU & very rarely with I/O devices is called as CPU bound processes.
- I/O bound processes - The process which spends more time in I/O operation than computation (time spends with CPU) is I/O bound processes.

Diagram:



- A process execution consists of cycle of CPU execution & I/O wait.
- A process starts its execution when CPU is assigned to it, so process execution begins with CPU burst cycle.
- This is followed by an I/O burst cycle when process is busy doing I/O operations.
- A process switch frequently from CPU burst cycle to I/O burst cycle & vice versa.
- The complete execution of process starts with CPU burst cycle, followed by I/O burst cycle, then followed by another CPU burst cycle, and so on.
- The final CPU burst cycle ends with system request to terminate execution.

■ Preemptive & Non-Preemptive Scheduling

- In preemptive scheduling, CPU is allocated to processes for limited time. The executing process in preemptive scheduling is interrupted in middle of execution when higher priority one comes.
- In non-preemptive scheduling, the CPU is allocated to process till it terminates or switches to waiting state. The executing process is not interrupted in middle of execution and waits till its execution.

- Difference:

Pre-emptive scheduling

- (i) Resource (CPU cycle) are allocated to process for limited time.
- (ii) Process can be interrupted in between.
- (iii) CPU utilization is high.
- (iv) Waiting time & response time is less.
- (v) eg: Round Robin.

Non-Preemptive scheduling

- Once resources are allocated to process, the process holds it till it is completed.
- Process can not be interrupted until it terminates itself.
- CPU utilization is low.
- Waiting time & response time is high.
- eg: FCFS

• SJF Algorithm

- SJF stands for Shortest Job first. It is both non-preemptive & preemptive algorithm.
- The process with shortest expected processing time is assigned to CPU first.
- Jobs/processes are processed in ascending order of their CPU burst times. Every time job with smallest CPU burst time is selected from ready queue.

Formulas

Wait-time = Start-time - Arrival time

TAT = End-time - Arrival time

Page no.: _____

Date: ____/____/____

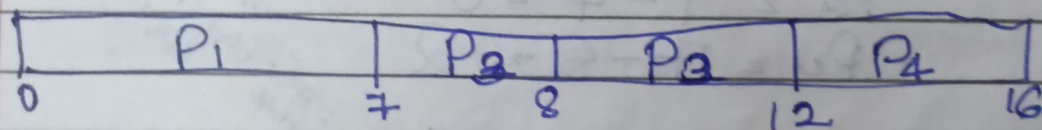
- If two processes having same CPU burst time then they will be scheduled according to FCFS algorithm.
- Average waiting time is less in SJF as compared to FCFS.
- SJF is probably optimal.
- The waiting time of longer burst time process is more.

* Example

① Process	Arrival time	Burst time
P ₁	0	7
P ₂	2	4
P ₃	4	1
P ₄	5	4

Non-preemptive.

Grantt Chart:



Waiting time:

$$P_1 = 0 - 0 = 0$$

$$P_2 = 8 - 2 = 6$$

$$P_3 = 7 - 4 = 3$$

$$P_4 = 12 - 5 = 7$$

$$\text{Avg} = (0 + 6 + 3 + 7) / 4 = 16 / 4 = 4$$

Turnaround time:

$$P_1 = 7 - 0 = 7$$

$$P_2 = 12 - 2 = 10$$

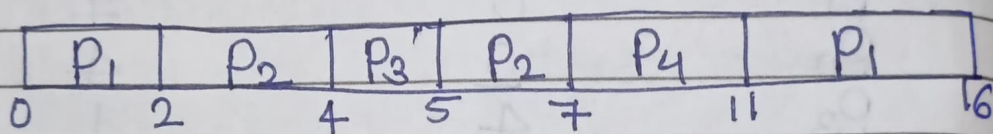
$$P_3 = 8 - 4 = 4$$

$$P_4 = 18 - 5 = 11$$

$$\text{Avg. TAT} = (7 + 10 + 4 + 11) / 4 = 32 / 4 = 8$$

② Preemptive

Gantt Chart:



Waiting time:

$$P_1 = (0 - 0) + (11 - \frac{7}{2}) = 0$$

$$P_2 = 2 - 2 = 0 + (5 - \frac{7}{2}) = 0$$

$$P_3 = 4 - 4 = 0$$

$$P_4 = 7 - 5 = 2$$

Start-time
End-time
for process

$$\text{Avg} = (0 + 0 + 0 + 2) / 4 = 2 / 4 = 0.5$$

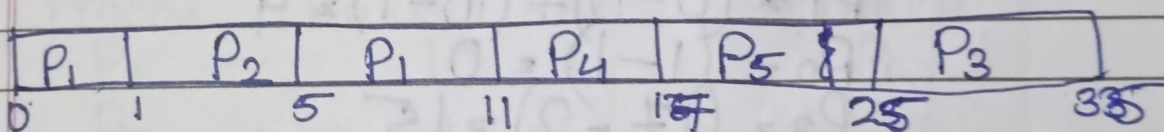
$$12 / 4 = 3$$

③ Preemptive SJF:

Process	Arrival Time	Burst Time
P ₁	0	7
P ₂	1	4
P ₃	2	10
P ₄	3	6
P ₅	4	8

~~Waiting time: Gantt Chart:~~

Gantt Chart:



Waiting time:

$$P_1 = (0-0) + (5-1) = 4$$

$$P_2 = (1-1) = 0$$

$$P_3 = 25 - 2 = 23$$

$$P_4 = 11 - 3 = 8$$

$$P_5 = 17 - 4 = 13$$

$$\text{Avg} = (4 + 0 + 23 + 8 + 13) / 5 = 48 / 5 = 9.6$$

Preemptive SJF is sometimes called shortest-remaining-time-first scheduling.

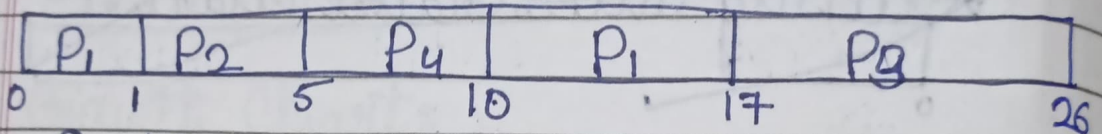
Page no.: _____

Date: ____/____/____

④ Pre-emptive

Process	Arrival Time	Burst Time
P ₁	0	8
P ₂	1	4
P ₃	2	9
P ₄	3	5

Gantt Chart:



Waiting time:

$$P_1 \rightarrow (0 - 0) + (10 - 1) = 9$$

$$P_2 \rightarrow (1 - 1) = 0$$

$$P_3 \rightarrow (17 - 2) = 15$$

$$P_4 \rightarrow (5 - 3) = 2$$

$$\text{Avg} = (9 + 15 + 2) / 4 = 26 / 4 = 6.5$$

Turnaround time:

$$P_1 \rightarrow (17 - 0) = 17$$

$$P_2 \rightarrow$$

$$P_3 \rightarrow$$

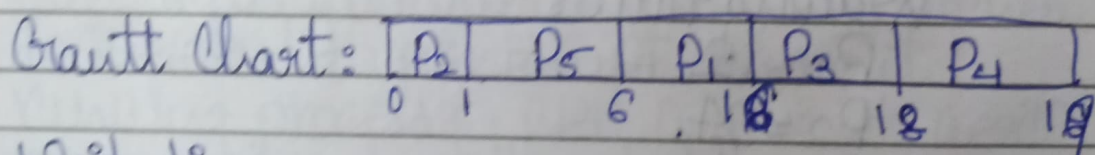
$$P_4 \rightarrow$$

Priority Scheduling

- A priority is associated with each process & scheduler always picks up highest priority process for execution from ready queue.
- Equal priority processes are scheduled in FCFS order.
- It can be either pre-emptive or non-preemptive.
- If priority of newly arrived process is higher than priority of currently running process, the process which was allocated is interrupted & return to queue.

①

Process	Burst Time	Priority
P ₁	10	3
P ₂	1	1
P ₃	2	3
P ₄	1	4
P ₅	5	2



Wait time:

$$P_1 = 6$$

$$P_2 = 0$$

$$P_3 = 16$$

$$P_4 = 18$$

$$P_5 = 1$$

$$\text{Avg} = (6 + 0 + 16 + 18 + 1) / 5$$

$$= 41/5$$

$$= 8.2 \text{ msec}$$

Turnaround time:

$$P_1 = 16$$

$$P_2 = 1$$

$$P_3 = 18$$

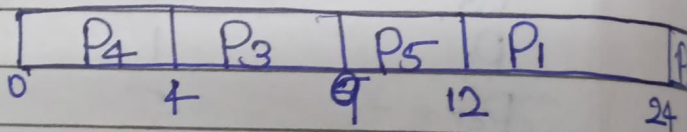
$$P_4 = 19$$

$$P_5 = 6$$

$$\begin{aligned} \text{Avg} &= (1+16+18+19+6)/5 \\ &= 60/5 \\ &= 12 \end{aligned}$$

Process	Burst time	Priority
P ₁	12	4
P ₂	10	5
P ₃	5	2
P ₄	4	1
P ₅	3	3

Gantt Chart:
Waiting time



Waiting time:

$$P_1 = 12$$

$$P_2 = 24$$

$$P_3 = 4$$

$$P_4 = 0$$

$$P_5 = 9$$

$$\text{Avg} = (12+24+4+0+9)/5 = 49/5 = 9.8$$

TAT:

$$P_1 = 24$$

$$P_2 = 34$$

$$P_3 = 9$$

$$P_4 = 4$$

$$P_5 = 12$$

$$\begin{aligned} \text{Avg} &= (24+34+9+4+12)/5 = 83/5 \\ &= 16.6 \end{aligned}$$

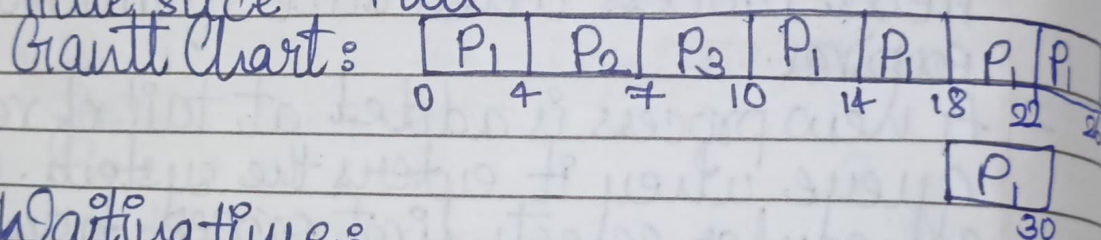
- Round Robin (RR) Scheduling algorithm.
- It is a preemptive scheduling algorithm. A small unit of time is known as time quantum/time slice is used for pre-emption of currently running process.
- Ready queue is implemented as a circular queue. CPU is assigned to entire processes one by one, on first come first serve basis for a specific time period.
- Every process executes for specified time period and CPU is given to next process when time quantum expires.
- A new process is added at tail of ready queue when it enters the system. CPU scheduler selects first process from head of ready queue and executes it for specified time quantum.
- Once time quantum expires, dispatcher is invoked to pre-empt current running process & CPU is given to next process placed at head of ready queue.
- The running process may have a CPU burst time less or greater than time quantum. If burst time of running process is less than time quantum then, process itself releases the CPU.
- The scheduler then selects next process from ready queue & executes it.

- If burst time of running process is longer than time quantum then, context switch occurs and process is placed at tail of ready queue for remaining burst time execution.

Example:

Process	Burst time
P ₁	24
P ₂	3
P ₃	3

Time slice - 4 ms



Waiting time:

$$P_1 = 0 + (10 - 4) = 6$$

$$P_2 = 4$$

$$P_3 = 7$$

$$\text{Avg} = (6 + 4 + 7) / 3 = 17 / 3 = 5.66 \text{ ms}$$

Turnaround time:

$$P_1 = 30$$

$$P_2 = 7$$

$$P_3 = 10$$

$$\text{Avg} = (30 + 7 + 10) / 3 = 47 / 3 = 15.6 \text{ ms}$$

▪ Deadlock

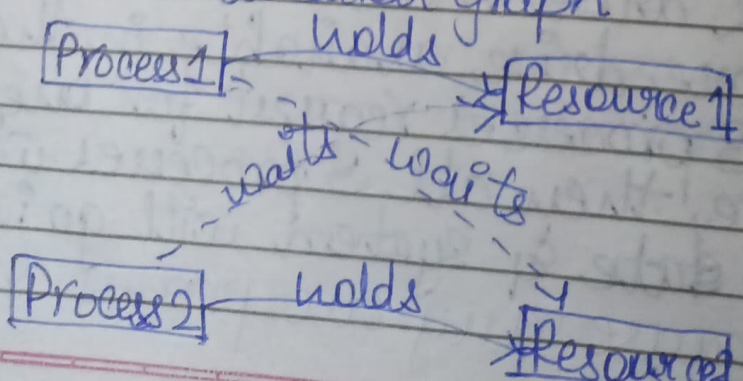
- It is a situation when two or more processes get locked & cannot proceed further because of interdependability.
- In multiprogramming environment, several processes may compete for a finite number of resources.
- A process requests resources and if resources are not available then process enters into waiting state.
- Sometimes a waiting process is never again able to change its status because the resources requested by it are held by other waiting processes. This situation is called as deadlock.
- When a process request for resources held by another waiting process which in turn is waiting for resources held by another waiting process and not a single process can execute its task, then deadlock occurs in the system.
- Example: Consider a system with 3 disk drives and 3 processes. When each process request one disk drive, system allocates one disk drive to each process. Now there is no more drive available in the system. If all 3 processes request for one more disk drive, then all 3 processes will go into waiting state & system will go in deadlock state. Because any one process from three

can execute only when one of them will release the disk drive allocated to it.

- Necessary conditions:

- i) Mutual exclusion - At least one resource must be held in a non-shareable mode; that is, only one process at a time can use the resource.
- ii) Hold & wait - A process must be holding at least one resource and waiting to acquire additional resources that are currently being held by other processes.
- iii) No pre-emption - Resources cannot be pre-empted i.e. a resource can be released only voluntarily by process holding it.
- iv) Circular wait - A set $\{P_0, P_1, \dots, P_n\}$ of waiting processes must exist such that P_0 is waiting for a resource held by P_n , P_1 is waiting for a resource held by P_2 , ..., P_{n-1} is waiting for resource held by P_n & P_n is waiting for a resource held by P_0 . Each process is waiting for resources held by other waiting processes in circular form.

Resource allocation graph



- Sequence when a process may utilize a resource:

- a) Request - If it is not possible to fulfil the request instantly, then requesting process must wait until it can get the resource.
- b) Use - The process use the resource to accomplish task.
- c) Release - The process free the resource after use.

- Deadlock handling

A deadlock in operating system can be handled in following 4 different ways:

- ① Adopt methods for avoiding the deadlock
- ② Prevent the deadlock from occurring
- ③ Ignore the deadlock
- ④ Allow deadlock to occur, detect it & recover from it.

- Deadlock prevention

- ① Mutual exclusion - This condition must hold for non-sharable resources. Sharable resources do not require mutually exclusive access, thus cannot be involved in deadlock.
- ② Hold & wait - One way to avoid this hold & wait is when a process requests a resource; it does not hold any other resources. One protocol that can be used requires each process to request & be allocated all its resources.

before it begins execution. Another protocol that can be used is, to allow a process to request resources only when process has no. A process may request some resources and use them. Before it requests any additional resources, it must release all resources that are currently allocated to it.

(iii) No pre-emption - If a process that is holding some resources requests another resource that cannot be immediately allocated to it then all resources currently being held are preempted. That is these resources are implicitly released. The preempted resources are added to list of resources for which process is waiting. Process will be restarted only when all resources i.e. its old resources as well as new ones that it is requesting will be available.

(iv) Circular wait - Circular-wait condition never holds is to impose a total ordering of all resource types, and to require that each process requests resources in an increasing order of enumeration. Let $R = \{R_1, R_2, \dots, R_n\}$ be the set of resource types. We assign to each resource type a unique integer number, which allows us to compare two resources R_i to determine whether one precedes another in our ordering. Formally, define a one-to-one function $f: R \rightarrow N$, where N is

set of natural numbers.

• Banker's Algorithm.

① This algorithm calculates resources allocated, required & available before allocating resources to any process to avoid deadlock. It contains two matrices on dynamic basis.

② Matrix A contains resources allocated to different processes at a given time. Matrix B maintains resources which are still required by different processes at same time.

Algorithm F: free resources.

Step 1: When process requests for a resource, the OS allocates it on a trial basis.

Step 2: After trial allocation, OS updates matrices & vectors. This updating can be done by OS in separate work area in memory.

Step 3: It compares F vector with each row of matrix B on vector to vector basis.

Step 4: If F is smaller than each of the row in Matrix B i.e. even if all free resources are allocated to any process in Matrix B & not a single process can complete its task then OS concludes that system is in unstable state.

no state

Step 5: If F is greater than any row for a process in Matrix B the OS allocates all required resources for that process on trial basis. It assumes that after completion of process, it will release all resources allocated to it. These resources can be added to free vector.

Step 6: After execution of process, it removes row indicating executed process from both matrices

Step 7: This algorithm will repeat procedure step 3 for each process from matrices & finds that all processes can complete execution without entering unsafe state. For each request for any resource by a process OS goes through all these trials of imaginary allocation & updation. After this if system remains in safe state and then changes can be made in actual matrices